

贪心的商人

题目描述

商人经营一家店铺，有 $number$ 种商品，由于仓库限制每件商品的`最大持有数量`是 $item[index]$ ，每种商品的价格是 $item_price[item_index][day]$ 。

通过对商品的买进和卖出获取利润，请给出商人在 $days$ 天内能获取的最大的利润。

注：同一件商品可以反复买进和卖出

输入描述

3 // 输入商品的数量 $number$

3 // 输入商人售货天数 $days$

4 5 6 // 输入仓库限制每件商品的`最大持有数量`是 $item[index]$

1 2 3 // 输入第一件商品每天的价格

4 3 2 // 输入第二件商品每天的价格

1 5 3 // 输入第三件商品每天的价格

输出描述

32 // 输出商人在这段时间内的最大利润

备注

根据输入的信息：

```
number = 3
days = 3
item[3] = {4, 5, 6}
item_price[3][4] = {{1, 2, 3}, {4, 3, 2}, {1, 5, 3}}
```

- 针对第一件商品，商人在第一天的价格是 $\text{item_price}[0][0] = 1$ 时买入 $\text{item}[0]$ 件，在第三天 $\text{item_price}[0][2] = 3$ 的时候卖出，获利最大是 8；
- 针对第二件商品，不进行交易，获利最大时 0；
- 针对第三件商品，商人在第一天价格是 $\text{item_price}[2][0] = 1$ 时买入 $\text{item}[2]$ 件，在第二天 $\text{item_price}[2][1] = 5$ 的时候卖出，获利最大是24；

因此这段时间商人能获取的最大利润是 $8 + 24 = 32$ ；

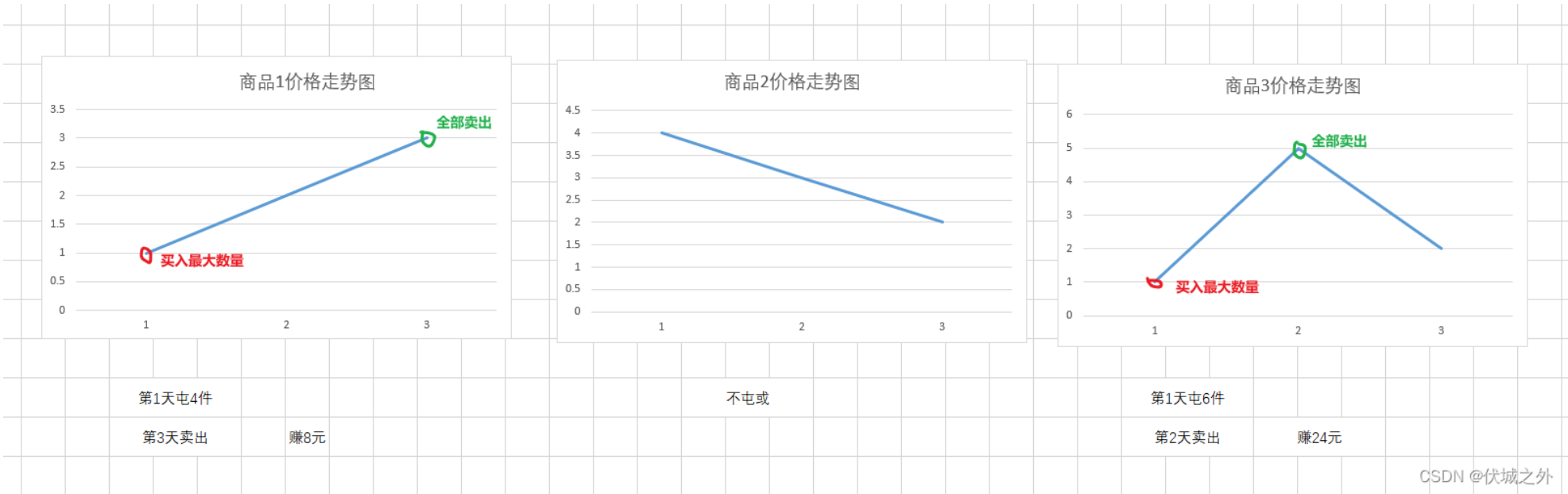
用例

输入	3 3 4 5 6 1 2 3 4 3 2 1 5 2
输出	32
说明	无

输入	1 1 1 1
输出	0
说明	无

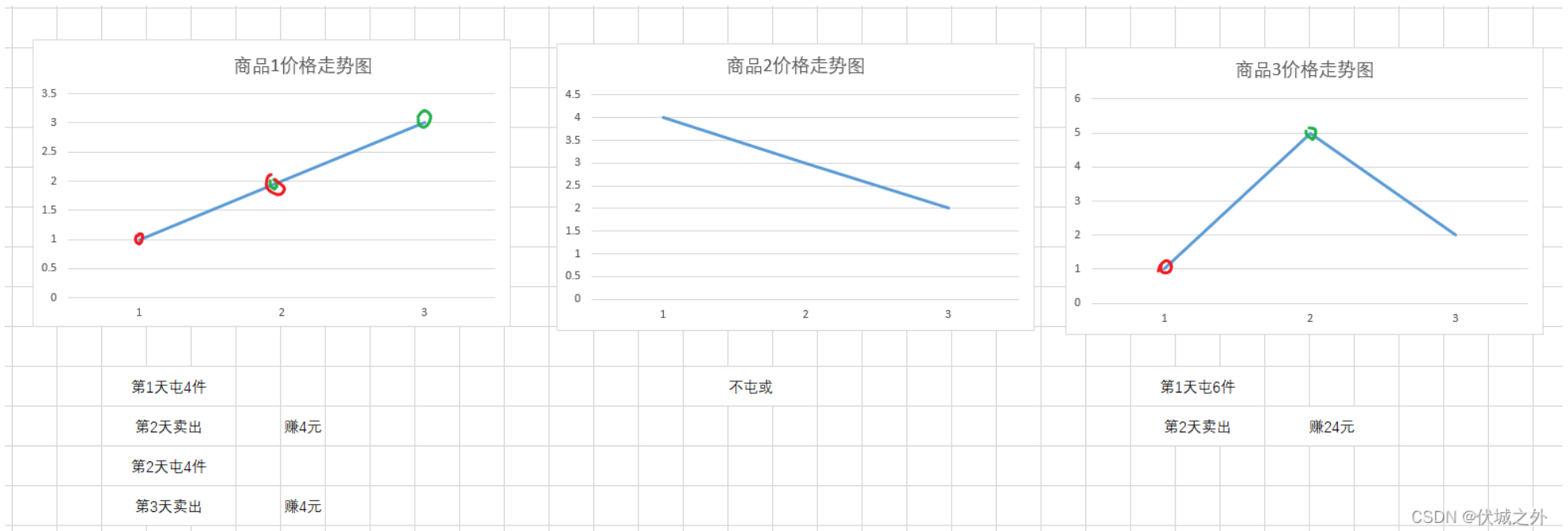
题目解析

用例含义如下所示



因此最多可以赚 $8 + 24 = 32$ 元。

另外，本题描述说：同一件商品可以反复买进和卖出，因此上面图示买卖操作还可以变为这样：



我们只要找到商品价格走势的上升区段，然后低价all in买入，高价all out卖出即可求得最大利润。

那么如何找到上升区段呢？

我们假设商品1的第 i 天的价格为 $price1[i]$ ，那么只要 $price1[i] < price1[i+1]$ ，则说明当前处于上升区段的低价位，因此可以all in，然后到 $i+1$ 天的时候all out。

JS算法源码

```
1 const rl = require("readline").createInterface({ input: process.stdin });
2 var iter = rl[Symbol.asyncIterator]();
3 const readline = async () => (await iter.next()).value;
4
5 void (async function () {
6   // 商品的数量
7   const number = parseInt(await readline());
8   // 商人售货天数
9   const days = parseInt(await readline());
```

运行

```

10 // 仓库限制每件商品的最大持有数量
11     const item = (await readline()).split(" ").map(Number);
12
13 // 商品每天的价格
14 const item_price = [];
15 for (let i = 0; i < number; i++) {
16     item_price.push((await readline()).split(" ").map(Number));
17 }
18
19 let ans = 0;
20
21 for (let i = 0; i < number; i++) {
22     for (let j = 0; j < days - 1; j++) {
23         ans += Math.max(item_price[i][j + 1] - item_price[i][j], 0) * item[i];
24     }
25 }
26
27 console.log(ans);
28 })();

```

Java算法源码

```

1 import java.util.Scanner;
2
3 public class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         int number = sc.nextInt();
8         int days = sc.nextInt();
9
10        int[] item = new int[number];
11        for (int i = 0; i < number; i++) {
12            item[i] = sc.nextInt();

```

```

13     }
14     int[][] item_price = new int[number][days];
15     for (int i = 0; i < number; i++) {
16         for (int j = 0; j < days; j++) {
17             item_price[i][j] = sc.nextInt();
18         }
19     }
20
21
22     int ans = 0;
23
24     for (int i = 0; i < number; i++) {
25         for (int j = 0; j < days - 1; j++) {
26             ans += Math.max(item_price[i][j + 1] - item_price[i][j], 0) * item[i];
27         }
28     }
29
30     System.out.println(ans);
31 }
32 }

```

Python算法源码

```

1  if __name__ == '__main__':
2      number = int(input())
3      days = int(input())
4      item = list(map(int, input().split()))
5      item_price = [list(map(int, input().split())) for _ in range(number)]
6
7      ans = 0
8
9      for i in range(number):
10         for j in range(days - 1):
11             ans += max(item_price[i][j + 1] - item_price[i][j], 0) * item[i]
12

```

```
13 |     print(ans)
```

C算法源码

```
1  #include <stdio.h>
2  #include <math.h>
3
4  int main() {
5      int number, days;
6      scanf("%d %d", &number, &days);
7
8      int item[number];
9      for (int i = 0; i < number; i++) {
10         scanf("%d", &item[i]);
11     }
12
13     int item_price[number][days];
14     for (int i = 0; i < number; i++) {
15         for (int j = 0; j < days; j++) {
16             scanf("%d", &item_price[i][j]);
17         }
18     }
19
20     int ans = 0;
21
22     for (int i = 0; i < number; i++) {
23         for (int j = 0; j < days - 1; j++) {
24             ans += (int) fmax(item_price[i][j + 1] - item_price[i][j], 0) * item[i];
25         }
26     }
27
28     printf("%d\n", ans);
29
30     return 0;
```

C++ 算法源码

```
1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main() {
6      int number, days;
7      cin >> number >> days;
8
9      vector<int> item(number);
10     for (int i = 0; i < number; i++) {
11         cin >> item[i];
12     }
13
14     vector<vector<int>> item_price(number, vector<int>(days));
15     for (int i = 0; i < number; i++) {
16         for (int j = 0; j < days; j++) {
17             cin >> item_price[i][j];
18         }
19     }
20
21     int ans = 0;
22
23     for (int i = 0; i < number; i++) {
24         for (int j = 0; j < days - 1; j++) {
25             ans += max(item_price[i][j + 1] - item_price[i][j], 0) * item[i];
26         }
27     }
28
29     cout << ans << endl;
30
31     return 0;
```

[复制](#)

